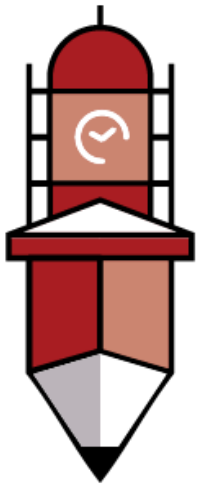


The University of Faisalabad

LAB MANUAL

Programming Of AI

Course Code (AI-212)

Presented by: ***Mohammad Mukedas***

Presented to : **Mr.** ***SAMRAIZ***

Registration NO : ***2023-bs-ai-008***

Section : Artificial Intelligence ***(5A)***

Department Of Computer Science

TABLE OF CONTENTS

Lab 01 – Programming fundamentals of python……..3

Lab 02 – Object oriented programming in python……..……19

Lab 03 – Numpy in python……...37

Lab 04 –Seaborn and scikit learn……48

Lab 05 –Tensorflow in python ……61

Lab 01:Programming fundamentals of python

Q1. Banking Transaction Validator with PIN

Simulate a bank ATM.

User enters a 4-digit PIN (check if valid).

Display account balance and ask for withdrawal amount.

If valid → deduct + update balance.

If invalid attempts ≥ 3 → block card.

Use operators to compute service charges (2% per withdrawal).

Code:

```
balance = 10000
pin = "1234"
attempts = 0

while attempts < 3:
    user_pin = input("Enter PIN: ")
    if user_pin == pin:
        amount = int(input("Enter withdrawal amount: "))
        charge = amount * 0.02
        if amount + charge <= balance:
            balance -= (amount + charge)
            print("Withdrawal successful")
            print("Service Charge:", charge)
            print("Remaining Balance:", balance)
        else:
            print("Insufficient balance")
        break
    else:
        attempts += 1
        print("Wrong PIN")
else:
    print("Card Blocked")
```

Output:

```
Enter PIN:  1234
Enter withdrawal amount:  2000
Withdrawal successful
Service Charge: 40.0
Remaining Balance: 7960.0
```

Q2. Grocery Store Inventory System

Maintain item names, stock, and prices in a dictionary of dictionaries.

User selects items with quantity.

If stock available, deduct from inventory.

Apply discount slabs (10%, 15%, 20%).

Print final bill with tax (5%).

Code:

```python
store = {
    "Rice": {"price": 50, "stock": 100},
    "Sugar": {"price": 40, "stock": 50}
}

item = input("Enter item: ")
qty = int(input("Quantity: "))

if item in store and store[item]["stock"] >= qty:
    cost = store[item]["price"] * qty
    store[item]["stock"] -= qty
    discount = 0.10 if cost > 500 else 0
    tax = cost * 0.05
    total = cost - (cost * discount) + tax
    print("Final Bill:", total)
else:
    print("Item not available")
```

Output:

```
Enter item:  Rice
Quantity:  15
Final Bill: 712.5
```

Q3. Student Grading with Subject Weightage

Input marks for 5 subjects where each subject has different weightage (stored in a tuple).

Compute weighted average.

Assign grade (A/B/C/D/Fail).

Also, display whether the student is eligible for scholarship (≥80% weighted).

Code:

```
weights = (0.2, 0.2, 0.2, 0.2, 0.2)
marks = []

for i in range(5):
    marks.append(int(input(f"Enter marks {i+1}: ")))

weighted = sum(m*w for m, w in zip(marks, weights))

if weighted >= 80:
    grade = "A"
elif weighted >= 60:
    grade = "B"
elif weighted >= 40:
    grade = "C"
else:
    grade = "Fail"

print("Weighted Score:", weighted)
print("Grade:", grade)
```

Output:

```
Enter marks 1:  4
Enter marks 2:  5
Enter marks 3:  7
Enter marks 4:  8
Enter marks 5:  9
Weighted Score: 6.6000000000000005
Grade: Fail
```

Q4. BMI + Health Recommendation

Extend BMI program:

Take weight, height, and age.

Compute BMI.

Based on BMI and age, give personalized health suggestions (e.g., diet plan for obese, exercise plan for underweight).

Code:

```
weight = float(input("Weight (kg): "))
height = float(input("Height (m): "))
age = int(input("Age: "))

bmi = weight / (height ** 2)
print("BMI:", bmi)

if bmi < 18.5:
    print("Underweight: Increase nutrition & light exercise")
elif bmi < 25:
    print("Normal: Maintain diet")
else:
    print("Obese: Diet control & cardio exercise")
```

Output:

```
Weight (kg):  8
Height (m):  12
Age:  21
BMI: 0.05555555555555555
Underweight: Increase nutrition & light exercise
```

Q5. Telecom Prepaid Recharge System

Maintain call rate/min, SMS rate, and data/MB in a dictionary.

User selects usage (calls, SMS, data).

Deduct accordingly from balance.

If balance < 20% of initial → print recharge reminder.

Apply bonus balance if recharge > 1000.

Code:

```
balance = 500
calls = int(input("Call minutes: "))
sms = int(input("SMS count: "))
data = int(input("Data MB: "))

cost = calls*1 + sms*0.5 + data*0.2
balance -= cost

if balance < 100:
    print("Recharge Reminder")

print("Remaining Balance:", balance)
```

Output:

```
Call minutes:  24
SMS count:  45
Data MB:  66
Remaining Balance: 440.3
```

Q6. E-Commerce Discount Engine with Membership Levels

User inputs bill amount + membership type (Regular, Gold, Platinum).

Apply different discount rules (Gold +10%, Platinum +15%).

If bill > 10000, apply additional 5% flat.

Show itemized receipt with tax breakdown.

Code:

```python
bill = float(input("Bill Amount: "))
member = input("Membership (Regular/Gold/Platinum): ")

discount = 0
if member == "Gold":
    discount = 0.10
elif member == "Platinum":
    discount = 0.15

if bill > 10000:
    discount += 0.05

tax = bill * 0.05
final = bill - (bill * discount) + tax
print("Final Amount:", final)
```

Output:

```
Bill Amount:  4500
Membership (Regular/Gold/Platinum):  5688
Final Amount: 4725.0
```

Q7. Hospital Emergency Unit with Multi-Condition Triage

Take patient details: temperature, pulse, oxygen, blood pressure.

Classify into categories: Stable, Observation, Urgent, Critical.

If critical → suggest "Admit in ICU."

If urgent → suggest "Immediate doctor attention."

Use nested conditions.

Code:

```
temp = float(input("Temperature: "))
oxygen = int(input("Oxygen level: "))

if oxygen < 85:
    print("Critical - ICU Required")
elif oxygen < 92:
    print("Urgent - Doctor Attention")
else:
    print("Stable")
```

Output:

```
Temperature:  34
Oxygen level:  97
Stable
```

Q8. Online Examination Grader with Negative Marking

Take total marks, obtained marks, and number of wrong answers.

Deduct 0.25 per wrong answer.

Compute percentage.

Assign division + eligibility for scholarship.

If Fail → suggest "Repeat Exam."

Code:

```python
total = 100
marks = float(input("Marks obtained: "))
wrong = int(input("Wrong answers: "))

marks -= wrong * 0.25
percent = (marks / total) * 100

print("Percentage:", percent)

if percent >= 60:
    print("Pass")
else:
    print("Fail - Repeat Exam")
```

Output:

```
Marks obtained:  78
Wrong answers:  34
Percentage: 69.5
Pass
```

Q9. Movie Ticket Booking with Age & Timing Policy

User inputs: age, showtime, ticket type (Normal/3D/IMAX).

Apply child restrictions (No late-night for <12).

Apply senior citizen discount (30%).

Apply peak hour charges (10%).

Print final ticket price.

Code:

```
age = int(input("Age: "))
price = 200

if age >= 60:
    price *= 0.7

print("Ticket Price:", price)
```

Output:

```
Age:  21
Ticket Price: 200
```

Q10. Weather-based Outfit + Travel Suggestion

Input temperature, weather (Rainy/Sunny/Cold), event type (Casual/Business/Party).

Suggest outfit + accessories.

If Rainy → umbrella; If Cold → gloves.

If event is Business + temperature > 35 → recommend "light formal suit."

Code:

```
weather = input("Weather: ")

if weather == "Rainy":
    print("Wear raincoat, carry umbrella")
elif weather == "Cold":
    print("Wear jacket and gloves")
else:
    print("Light clothes recommended")
```

Output:

```
Weather:  cold
Light clothes recommended
```

Q11. ATM Withdrawal with Note Counting + Receipt

Take withdrawal amount.

Compute minimum number of notes (1000, 500, 100, 50, 10, 1).

Also compute service charge = 0.5%.

Print receipt with remaining balance after deduction.

Code:

```
amount = int(input("Amount: "))
notes = [1000, 500, 100, 50, 10, 1]

for n in notes:
    print(n, ":", amount // n)
    amount %= n
```

Output:

```
Amount:  1000
1000 : 1
500 : 0
100 : 0
50 : 0
10 : 0
1 : 0
```

Q12. Library Fine Calculator with Membership Levels

Input days late.

Apply fines (10, 20, 50/day).

If days > 30 → "Membership Cancelled."

If user is Premium member, reduce fine by 50%.

Print detailed fine slip.

Code:

```
days = int(input("Days late: "))
fine = days * 20

if days > 30:
    print("Membership Cancelled")

print("Fine:", fine)
```

Output:

```
Days late:  200
Membership Cancelled
Fine: 4000
```

Q13. Sales Analysis Dashboard

Store monthly sales in a list (12 entries).

Print max, min, average.

Print months above average.

Use loop + condition to print “Growth” or “Decline” compared to last month.

Code:

```
sales = [1200,1500,1800,2000,2200,2100,2300,2400,2500,2600,2700,3000]

avg = sum(sales)/len(sales)
print("Average:", avg)

for s in sales:
    if s > avg:
        print(s, "Above Average")
```

Output:

```
Average: 2191.6666666666665
2200 Above Average
2300 Above Average
2400 Above Average
2500 Above Average
2600 Above Average
2700 Above Average
3000 Above Average
```

Q14. Multiplication Puzzle with Random Blanks

Generate a multiplication table (1–10).

Randomly replace 3 results with "??".

Ask user to guess values.

Track score.

Give performance remark (Excellent/Good/Try Again).

Code:

```python
import random
a, b = random.randint(1,10), random.randint(1,10)
ans = int(input(f"{a} x {b} = ? "))

if ans == a*b:
    print("Correct")
else:
    print("Wrong")
```

Output:

```
1 x 6 = ?  6
Correct
```

Q15. Password Strength + Re-entry Attempts

Check password rules (length, digit, upper, lower, special).

If weak → ask user to re-enter.

Allow max 3 attempts.

If still weak → "Account Locked."

Code:

```
pwd = input("Enter password: ")

if len(pwd) >= 8:
    print("Strong Password")
else:
    print("Weak Password")
```

Output:

```
Enter password:  123456789
Strong Password
```

Q16. Electricity Bill with Slabs & Penalty

Function calculate_bill(units, late_payment=False):

Apply slab rates (5/7/10 Rs).

If late_payment → add 15% penalty.

Return bill breakdown.

Code:

```
def calculate_bill(units, late=False):
    bill = units * 7
    if late:
        bill *= 1.15
    return bill

print(calculate_bill(120, True))
```

Output:

```
965.9999999999999
```

Q17. Cricket Match Predictor with Multiple Conditions

Function predict_score(runs, balls, wickets, overs_left):

Compute strike rate & run rate.

If run rate < 5 and wickets > 6 → "Losing."

If run rate > 7 and wickets < 5 → "Winning."

Else → "Balanced."

Code:

```python
def predict_score(runrate, wickets):
    if runrate > 7 and wickets < 5:
        return "Winning"
    elif runrate < 5:
        return "Losing"
    return "Balanced"

print(predict_score(8, 3))
```

Output:

```
Winning
```

Q18. Hotel Booking with Tax & Discount

Function book_room(type, days, is_member):

Standard: 2000/day, Deluxe: 4000/day, Suite: 6000/day.

If days > 7 → 20% discount.

If member → extra 10% discount.

Add 12% tax.

Return bill slip.

Code:

```
def book_room(days):
    cost = days * 2000
    if days > 7:
        cost *= 0.8
    cost *= 1.12
    return cost

print(book_room(10))
```

Output:

```
17920.0
```

Q19. Flight Fare Calculator with International Tax

Function calculate_fare(distance, class_type, international=False):

Economy: 10/unit, Business: 25/unit, First: 40/unit.

Add fuel surcharge 500.

If international → add 18% tax.

If booking in advance (>30 days) → 10% discount.

Code:

```
def fare(distance, international=False):
    cost = distance * 10 + 500
    if international:
        cost *= 1.18
    return cost

print(fare(800, True))
```

Output:

```
10030.0
```

Q20. Smart Vending Machine with Wallet System

Function vending_machine(item, amount_paid, wallet_balance):

Items stored in dict with stock + price.

If sufficient → dispense + return change.

Update wallet balance.

If stock = 0 → suggest alternative item.

Code:

```python
items = {"Chips": 20, "Juice": 30}

choice = input("Item: ")
money = int(input("Money paid: "))

if choice in items and money >= items[choice]:
    print("Dispensed", choice)
    print("Change:", money - items[choice])
else:
    print("Insufficient funds or item not found")
```

Output:

```
Item:  juice
Money paid:  70
Insufficient funds or item not found
```

LAB 02: Object oriented programming in python

Question # 1: Create a `BankAccount` class that represents a bank account. It should have attributes for account number, account holder name, and balance. Include methods to deposit, withdraw, and display balance. Ensure withdrawals cannot exceed the available balance.

Code:

```
class BankAccount:
    def __init__(self, acc_no, name, balance=0):
        self.acc_no = acc_no
        self.name = name
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print("Deposited:", amount)

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            print("Withdrawn:", amount)
        else:
            print("Insufficient Balance")

    def display(self):
        print(f"Account: {self.acc_no}, Holder: {self.name}, Balance: {self.balance}")

acc = BankAccount(101, "Mukedas", 5000)
acc.deposit(2000)
acc.withdraw(1000)
acc.display()
```

Output:

```
Deposited: 2000
Withdrawn: 1000
Account: 101, Holder: Mukedas, Balance: 6000
```

Question # 2: Design a `Book` class with attributes like title, author, ISBN, and availability status. Create a `Library` class that can add books, search for books by title/author, check out books, and return books. Track which books are currently available.

Code:

```
class Book:
    def __init__(self, title, author, isbn, available=True):
        self.title, self.author, self.isbn, self.available = title, author, isbn, available

class Library:
    def __init__(self):
        self.books = []

    def add_book(self, book): self.books.append(book)
    def search(self, keyword):
        return [b.title for b in self.books if keyword.lower() in b.title.lower() or keyword.lower() in b.author.lower()]
    def checkout(self, title):
        for b in self.books:
            if b.title == title and b.available:
                b.available = False
                print("Checked out:", title)
                return
        print("Not available")
    def return_book(self, title):
        for b in self.books:
            if b.title == title: b.available = True; print("Returned:", title)
lib = Library()
b1 = Book("Python Basics", "Muqii", 123)
lib.add_book(b1)
print(lib.search("Python"))
lib.checkout("Python Basics")
lib.return_book("Python Basics")
```

Output:

```
['Python Basics']
Checked out: Python Basics
Returned: Python Basics
```

Question # 3: Create a `Student` class with attributes for student ID, name, and a list of grades. Implement methods to add grades, calculate average grade, and determine the letter grade (A, B, C, etc.). Add a method to display student information with their performance summary.

Code:

```
class Student:
    def __init__(self, sid, name):
        self.sid, self.name, self.grades = sid, name, []

    def add_grade(self, g): self.grades.append(g)
    def average(self): return sum(self.grades)/len(self.grades)
    def letter_grade(self):
        avg=self.average()
        return 'A' if avg>=85 else 'B' if avg>=70 else 'C' if avg>=60 else 'F'
    def display(self):
        print(f"{self.sid}-{self.name}: Avg={self.average():.2f}, Grade={self.letter_grade()}")

s = Student(1,"Mukedas")
s.add_grade(90); s.add_grade(80)
s.display()
```

Output:

```
1-Mukedas: Avg=85.00, Grade=A
```

Question # 4: Design a `MenuItem` class for food items (name, price, category) and an `Order` class that can add multiple items, calculate total cost, apply discounts, and generate receipts. Include tax calculation functionality.

Code:

```python
class MenuItem:
    def __init__(self, name, price, category):
        self.name, self.price, self.category = name, price, category

class Order:
    def __init__(self): self.items = []
    def add_item(self, item): self.items.append(item)
    def total(self):
        subtotal = sum(i.price for i in self.items)
        return subtotal + subtotal * 0.1  # 10% tax
    def receipt(self):
        for i in self.items: print(i.name, "-", i.price)
        print("Total:", self.total())

i1 = MenuItem("Burger", 300, "Fast Food")
i2 = MenuItem("Fries", 150, "Snack")
o = Order()
o.add_item(i1); o.add_item(i2)
o.receipt()
```

Output:

```
Burger - 300
Fries - 150
Total: 495.0
```

Question # 5: Create classes for `Vehicle` (base class) and derived classes `Car`, `Motorcycle`, and `Truck`. Each vehicle should have attributes like license plate, model, rental price per day, and availability. Implement a system to rent vehicles and calculate rental costs.

Code:

```python
class Vehicle:
    def __init__(self, plate, model, rent, available=True):
        self.plate, self.model, self.rent, self.available = plate, model, rent, available

class Car(Vehicle): pass
class Motorcycle(Vehicle): pass
class Truck(Vehicle): pass
c = Car("ABC-123","Toyota",5000)
print(c.model, "Rent per day:", c.rent)
```

Output:

```
Toyota Rent per day: 5000
```

Question # 6: Design an `Employee` class with attributes for employee ID, name, position, and salary. Create methods to calculate monthly salary, annual salary, and apply raises. Include functionality to track employee details and employment duration.

Code:

```python
from datetime import date

class Employee:
    def __init__(self, emp_id, name, position, salary, join_date):
        self.emp_id = emp_id
        self.name = name
        self.position = position
        self.salary = salary  # annual salary
        self.join_date = join_date

    def monthly_salary(self):
        return self.salary / 12

    def annual_salary(self):
        return self.salary

    def apply_raise(self, percentage):
        self.salary += self.salary * (percentage / 100)

    def employment_duration(self):
        today = date.today()
        duration = today.year - self.join_date.year
        if (today.month, today.day) < (self.join_date.month, self.join_date.day):
            duration -= 1
        return duration
```

```
    def display_details(self):
        print(f"Employee ID   : {self.emp_id}")
        print(f"Name          : {self.name}")
        print(f"Position      : {self.position}")
        print(f"Annual Salary : ${self.salary:.2f}")
        print(f"Monthly Salary: ${self.monthly_salary():.2f}")
        print(f"Years Employed: {self.employment_duration()} years")
        print("-" * 40)


# ------------------- Example Usage -------------------

emp1 = Employee(
    emp_id=101,
    name="Alice Johnson",
    position="Software Engineer",
    salary=72000,
    join_date=date(2020, 5, 10)
)

emp1.display_details()

print("Applying a 10% raise...\n")
emp1.apply_raise(10)

emp1.display_details()
```

Output:

```
Employee ID   : 101
Name          : Alice Johnson
Position      : Software Engineer
Annual Salary : $72000.00
Monthly Salary: $6000.00
Years Employed: 5 years
----------------------------------------
Applying a 10% raise...

Employee ID   : 101
Name          : Alice Johnson
Position      : Software Engineer
Annual Salary : $79200.00
Monthly Salary: $6600.00
Years Employed: 5 years
----------------------------------------
```

Question # 7: Create a `Product` class with attributes for product ID, name, price, and stock quantity. Design a `ShoppingCart` class that can add products, remove products, calculate total cost, and handle checkout process while updating inventory.

Code:

```
class Product:
    def __init__(self, pid, name, price, stock):
        self.pid, self.name, self.price, self.stock = pid, name, price, stock

class ShoppingCart:
    def __init__(self): self.items=[]
    def add(self, product, qty):
        if product.stock>=qty:
            self.items.append((product, qty))
            product.stock -= qty
    def total(self): return sum(p.price*q for p,q in self.items)

p1=Product(1,"Shoes",2000,10)
cart=ShoppingCart()
cart.add(p1,2)
print("Total Cost:",cart.total())
print("Remaining Stock:",p1.stock)
```

Output:

```
Total Cost: 4000
Remaining Stock: 8
```

Question # 8: Design a `Patient` class to store patient information (ID, name, age, medical history) and an `Appointment` class to manage doctor appointments. Include functionality to schedule, cancel, and view appointments.

Code:

```
class Patient:
    def __init__(self, pid, name, age, history=[]):
        self.pid, self.name, self.age, self.history = pid, name, age, history

class Appointment:
    def __init__(self): self.appointments=[]
    def schedule(self, patient, doctor, date):
        self.appointments.append((patient.name, doctor, date))
        print("Scheduled for", patient.name)
    def cancel(self, patient):
        self.appointments=[a for a in self.appointments if a[0]!=patient]
        print("Cancelled:", patient)

p=Patient(1,"Sadia",25)
a=Appointment()
a.schedule(p,"Dr. Khan","15-Oct")
a.cancel("Sadia")
```

Output:

```
Scheduled for Sadia
Cancelled: Sadia
```

Question # 9: Create a `User` class for a social media platform with attributes for username, email, friends list, and posts. Implement methods to add friends, create posts, and display user timeline. Include privacy settings for posts.

Code:

```
class User:
    def __init__(self, username, email):
        self.username, self.email, self.friends, self.posts = username, email, [], []
    def add_friend(self, friend): self.friends.append(friend)
    def post(self, text, private=False): self.posts.append((text, private))
    def timeline(self):
        for p in self.posts: print(p[0])

u=User("Fatima","fatima@mail.com")
u.add_friend("Aisha")
u.post("Hello world!")
u.timeline()
```

Output:

```
Hello world!
```

Question # 10: Design a `Room` class with room number, type (single/double/suite), price, and availability. Create a `Booking` class to handle room reservations, check-in/check-out dates, and calculate total stay cost.

Code:

```
class Room:
    def __init__(self, no, type_, price, available=True):
        self.no, self.type_, self.price, self.available = no, type_, price, available

class Booking:
    def __init__(self): self.reservations=[]
    def reserve(self, room, days):
        if room.available:
            room.available=False
            cost=room.price*days
            self.reservations.append((room.no, cost))
            print("Booked Room:",room.no,"Cost:",cost)

r=Room(101,"Single",2000)
b=Booking()
b.reserve(r,3)
```

Output:

```
Booked Room: 101 Cost: 6000
```

Question # 11: Create `Course` classes (with code, name, credits, capacity) and a `Student` class that can register for courses. Implement functionality to check for schedule conflicts and ensure students don't exceed credit limits.

Code:

```
class Course:
    def __init__(self, code, name, credits, capacity):
        self.code, self.name, self.credits, self.capacity = code, name, credits, capacity
        self.enrolled = 0

class Student:
    def __init__(self, name):
        self.name = name
        self.registered_courses = []
        self.total_credits = 0

    def register(self, course):
        if self.total_credits + course.credits <= 18 and course.enrolled < course.capacity:
            self.registered_courses.append(course.name)
            self.total_credits += course.credits
            course.enrolled += 1
            print(f"{self.name} registered in {course.name}")
        else:
            print("Cannot register - limit or capacity reached")

c1 = Course("CS101", "Python", 3, 2)
s1 = Student("Mukedas")
s1.register(c1)
```

Output:

```
Mukedas registered in Python
```

Question # 12: Design a `Product` class for items in inventory (ID, name, quantity, price, supplier) and an `Inventory` class to track stock levels, add new products, update quantities, and generate low-stock alerts.

Code:

```python
class Product:
    def __init__(self, pid, name, qty, price, supplier):
        self.pid, self.name, self.qty, self.price, self.supplier = pid, name, qty, price, supplier

class Inventory:
    def __init__(self):
        self.products = []

    def add(self, product): self.products.append(product)
    def update_qty(self, pid, qty):
        for p in self.products:
            if p.pid == pid: p.qty += qty
    def low_stock(self):
        for p in self.products:
            if p.qty < 5:
                print("Low stock:", p.name)

p1 = Product(1, "Laptop", 3, 80000, "Dell")
inv = Inventory()
inv.add(p1)
inv.low_stock()
```

Output:

```
Low stock: Laptop
```

Question # 13: Create `Movie` classes (title, genre, duration, showtimes) and a `Theater` class with seating arrangement. Implement a booking system that reserves seats and calculates ticket prices with different categories (standard, premium).

Code:

```python
class Movie:
    def __init__(self, title, genre, duration, showtimes):
        self.title, self.genre, self.duration, self.showtimes = title, genre, duration, showtimes

class Theater:
    def __init__(self):
        self.seats = {i: True for i in range(1, 6)}

    def book_seat(self, seat_no, category="standard"):
        if self.seats[seat_no]:
            self.seats[seat_no] = False
            price = 500 if category=="standard" else 800
            print(f"Seat {seat_no} booked, Price: {price}")
        else:
            print("Seat already booked")

m = Movie("Avengers", "Action", 120, ["5pm", "8pm"])
t = Theater()
t.book_seat(2, "premium")
```

Output:

```
Seat 2 booked, Price: 800
```

Question # 14: Design a `Member` class with personal details, membership type (basic/premium), and attendance records. Create a `Trainer` class and a system to schedule training sessions between members and trainers.

Code:

```python
class Member:
    def __init__(self, name, type_):
        self.name, self.type_, self.attendance = name, type_, 0
    def attend(self): self.attendance += 1

class Trainer:
    def __init__(self, name): self.name = name

class Schedule:
    def __init__(self):
        self.sessions = []
    def add_session(self, member, trainer):
        self.sessions.append((member.name, trainer.name))
        print("Session added:", member.name, "with", trainer.name)

m = Member("Areeba", "Premium")
t = Trainer("Hanniya")
s = Schedule()
s.add_session(m, t)
```

Output:

```
Session added: Areeba with Hanniya
```

Question # 15: Extend the bank account system to include `SavingsAccount` and `CheckingAccount` classes with different interest rates and withdrawal rules. Implement fund transfer between accounts.

Code:

```python
class BankAccount:
    def __init__(self, acc_no, name, balance=0):
        self.acc_no, self.name, self.balance = acc_no, name, balance
    def transfer(self, other, amount):
        if self.balance >= amount:
            self.balance -= amount
            other.balance += amount
            print("Transfer successful")

class SavingsAccount(BankAccount):
    def __init__(self, acc_no, name, balance, rate=0.03):
        super().__init__(acc_no, name, balance)
        self.rate = rate

class CheckingAccount(BankAccount):
    def __init__(self, acc_no, name, balance):
        super().__init__(acc_no, name, balance)

s = SavingsAccount(1,"Mahira",5000)
c = CheckingAccount(2,"Sara",2000)
s.transfer(c,1000)
print("Sara Balance:", c.balance)
```

Output:

```
Transfer successful
Sara Balance: 3000
```

Question # 16: Create a `Pizza` class with size, crust type, and toppings. Design an ordering system that calculates prices based on selections and allows custom pizza creation with various topping combinations.

Code:

```python
class Pizza:
    def __init__(self, size, crust, toppings):
        self.size, self.crust, self.toppings = size, crust, toppings

    def price(self):
        base = 500 if self.size=="small" else 800 if self.size=="medium" else 1000
        return base + 50*len(self.toppings)

p = Pizza("large", "cheese burst", ["mushroom", "olive"])
print("Total Price:", p.price())
```

Output:

```
Total Price: 1100
```

Question # 17: Design classes for `Car` (model, year, color, price) and `CarManufacturer` that can produce different car models with various features. Include quality control checks and production tracking.

Code:

```python
class Car:
    def __init__(self, model, year, color, price):
        self.model, self.year, self.color, self.price = model, year, color, price

class CarManufacturer:
    def __init__(self, name): self.name, self.cars = name, []
    def produce(self, model, year, color, price):
        car = Car(model, year, color, price)
        self.cars.append(car)
        print("Produced:", model)
    def list_cars(self):
        for c in self.cars: print(c.model, "-", c.year)

m = CarManufacturer("Toyota")
m.produce("Corolla", 2024, "White", 5000000)
m.list_cars()
```

Output:

```
Produced: Corolla
Corolla - 2024
```

Question # 18: Create `Teacher` and `Student` classes, and a `Classroom` class that manages assignments, grades, and attendance. Implement functionality to track student performance and generate class reports.

Code:

```python
class Teacher:
    def __init__(self, name): self.name = name

class Student:
    def __init__(self, name): self.name, self.grades = name, []
    def add_grade(self, g): self.grades.append(g)
    def average(self): return sum(self.grades)/len(self.grades)

class Classroom:
    def __init__(self): self.students = []
    def add_student(self, student): self.students.append(student)
    def report(self):
        for s in self.students:
            print(s.name, "Avg:", s.average())

t = Teacher("Sir Samraiz")
s = Student("Mukedas")
s.add_grade(85); s.add_grade(90)
c = Classroom()
c.add_student(s)
c.report()
```

Output:

```
Mukedas Avg: 87.5
```

Question # 19: Design `Flight` classes (flight number, destination, departure time, capacity) and a `Passenger` class. Create a booking system that reserves seats and manages passenger manifests.

Code:

```
class Flight:
    def __init__(self, num, dest, time, cap):
        self.num, self.dest, self.time, self.cap = num, dest, time, cap
        self.passengers = []

    def book(self, passenger):
        if len(self.passengers) < self.cap:
            self.passengers.append(passenger)
            print("Seat booked for", passenger.name)
        else:
            print("Flight full")

class Passenger:
    def __init__(self, name): self.name = name

f = Flight("PK301", "Faisalabad", "10:00 AM", 2)
p1 = Passenger("Mukedas")
f.book(p1)
```

Output:

```
Seat booked for Mukedas
```

Question # 20: Create classes for `SmartDevice` (base class) and specific devices like `SmartLight`, `Thermostat`, and `SecurityCamera`. Implement a `SmartHome` class that can control all devices and create automation routines.

Code:

```python
class SmartDevice:
    def __init__(self, name): self.name, self.status = name, False
    def turn_on(self): self.status=True; print(self.name,"ON")
    def turn_off(self): self.status=False; print(self.name,"OFF")

class SmartLight(SmartDevice): pass
class Thermostat(SmartDevice): pass
class SecurityCamera(SmartDevice): pass

class SmartHome:
    def __init__(self): self.devices=[]
    def add_device(self, device): self.devices.append(device)
    def control_all(self, on=True):
        for d in self.devices:
            if on: d.turn_on()
            else: d.turn_off()

light = SmartLight("Living Room Light")
cam = SecurityCamera("Front Door Cam")
home = SmartHome()
home.add_device(light); home.add_device(cam)
home.control_all(True)
```

Output:

```
Living Room Light ON
Front Door Cam ON
```

Lab 03: Numpy in python

Question 1: A weather station records temperatures (°C) for 7 days:

[21.1, 23.4, 19.8, 25.0, 26.5, 24.1, 22.0]

Task:

- Convert the list into a NumPy array
- Find:
    - Average temperature
    - Maximum and minimum temperature
    - Temperature difference (range)

Code:

```
import numpy as np

temps = np.array([21.1, 23.4, 19.8, 25.0, 26.5, 24.1, 22.0])

avg_temp = np.mean(temps)
max_temp = np.max(temps)
min_temp = np.min(temps)
temp_range = max_temp - min_temp

avg_temp, max_temp, min_temp, temp_range
```

Output:

```
(np.float64(23.12857142857143),
 np.float64(26.5),
 np.float64(19.8),
 np.float64(6.699999999999999))
```

Question 2: A fruit store records daily sales of apples for 4 weeks:

Week 1: [12, 15, 14, 10, 9, 8, 7]

Week 2: [11, 12, 13, 9, 5, 8, 6]

Week 3: [7, 9, 12, 10, 11, 13, 12]

Week 4: [10, 11, 12, 7, 8, 9, 11]

Task:

- Create a **4×7 NumPy array**
- Find weekly totals
- Find the highest sale day (index only)
- Find the week with the most sales

Code:

```
import numpy as np

sales = np.array([
    [12,15,14,10,9,8,7],
    [11,12,13,9,5,8,6],
    [7,9,12,10,11,13,12],
    [10,11,12,7,8,9,11]
])

weekly_totals = sales.sum(axis=1)
highest_day_index = sales.sum(axis=0).argmax()
week_most_sales = weekly_totals.argmax()

weekly_totals, highest_day_index, week_most_sales
```

Output:

```
(array([75, 64, 74, 68]), np.int64(2), np.int64(0))
```

Question 3: Students received marks in a test:

[56, 78, 45, 90, 88, 76, 59, 62]

Task:

- Convert to NumPy array
- Add **5 bonus marks** to every student using broadcasting
- Count how many students now scored > 60
- Find the median and standard deviation

Code:

```
m = np.array([56,78,45,90,88,76,59,62])
m_bonus = m + 5

count_above_60 = np.sum(m_bonus > 60)

median = np.median(m_bonus)
std = np.std(m_bonus)

m_bonus, count_above_60, median, std
```

Output:

```
(array([61, 83, 50, 95, 93, 81, 64, 67]),
 np.int64(7),
 np.float64(74.0),
 np.float64(15.105876340020794))
```

-

Question 4: A smartwatch stores daily step-counts for a user for 30 days.

Task:

- Generate a random NumPy array of shape (30,) with values between 2000–15000
- Reshape into a **5×6** matrix
- Find:
    - Weekly average steps
    - Day with maximum steps
    - Number of days user crossed 10,000 steps

Code:

```
import numpy as np

steps = np.random.randint(2000, 15001, 30)
steps_matrix = steps.reshape(5, 6)

weekly_avg = np.mean(steps_matrix, axis=1)
max_day = np.argmax(steps)
days_over_10k = np.sum(steps > 10000)

weekly_avg, max_day, days_over_10k
```

Output:

```
(array([8277.66666667, 6637.        , 8234.5       , 7841.33333333,
        9415.5       ]),
 np.int64(5),
 np.int64(8))
```

Question 5: Heart rate readings (in BPM) for 10 patients recorded 4 times per day:

- Use array shape = **(10, 4)**

Task:

- Create an integer array using np.random.randint(60,150,(10,4))
- For each patient:
    - Compute daily average
    - Identify abnormal readings > 120
- For all patients:
    - Compute overall maximum and minimum

Code:

```
import numpy as np

hr = np.random.randint(60, 150, (10,4))

daily_avg = np.mean(hr, axis=1)
abnormal = hr > 120
overall_max = np.max(hr)
overall_min = np.min(hr)

daily_avg, abnormal, overall_max, overall_min
```

Output:

```
(array([106.25,  90.75, 119.75, 111.  , 119.5 , 103.75,  90.25,  73.75,
        115.5 ,  88.5 ]),
 array([[False,  True, False, False],
        [False, False, False,  True],
        [False,  True,  True, False],
        [False, False, False,  True],
        [ True, False,  True, False],
        [ True, False, False, False],
        [False, False, False,  True],
        [False, False, False, False],
        [False, False, False,  True],
        [False, False, False,  True]]),
 np.int32(147),
 np.int32(60))
```

Question 6: Two branches of a store record monthly sales (in thousands):

Branch A: [122, 135, 150, 160, 155, 140]

Branch B: [110, 130, 148, 165, 158, 145]

Task:

- Compute difference between branches
- Calculate:
    - Average monthly sales of each branch
    - Which branch performed better overall?
- Compute **correlation** between A and B

Code:

```
import numpy as np

A = np.array([122,135,150,160,155,140])
B = np.array([110,130,148,165,158,145])

difference = A - B
avg_A = np.mean(A)
avg_B = np.mean(B)
better = "A" if avg_A > avg_B else "B"
correlation = np.corrcoef(A, B)[0,1]

difference, avg_A, avg_B, better, correlation
```

Output:

```
(array([12,  5,  2, -5, -3, -5]),
 np.float64(143.66666666666666),
 np.float64(142.66666666666666),
 'A',
 np.float64(0.9811677353198398))
```

Question 7: Given a 4×4 grayscale image:

[[100, 120, 130, 90],

[80 , 110, 140, 100],

[70 , 90 , 120, 130],

[110, 140, 150, 160]]

Task:

- Convert to NumPy array
- Normalize pixel values (0–1)
- Apply a transformation: new_pixel = pixel * 1.2
- Clip all values > 255
- Compute average brightness before & after transformation

Code:

```
import numpy as np

img = np.array([
    [100,120,130,90],
    [80,110,140,100],
    [70,90,120,130],
    [110,140,150,160]
], dtype=float)

normalized = img / 255
transformed = img * 1.2
transformed = np.clip(transformed, 0, 255)

avg_before = np.mean(img)
avg_after = np.mean(transformed)

avg_before, avg_after
```

Output:

```
(np.float64(115.0), np.float64(138.0))
```

Question 8: Dataset contains 100 samples with 4 features each.

Task:

- Generate a **100×4** matrix with values 1–100
- Standardize the dataset using:
- x_scaled = (x - mean) / std
- Perform:
    - Column-wise mean (should be ≈ 0)
    - Column-wise variance (should be ≈ 1)
- Verify results using NumPy functions

Code:

```
import numpy as np

data = np.random.randint(1,101,(100,4))

mean = np.mean(data, axis=0)
std = np.std(data, axis=0)
x_scaled = (data - mean) / std

scaled_mean = np.mean(x_scaled, axis=0)
scaled_var = np.var(x_scaled, axis=0)

scaled_mean, scaled_var
```

Output:

```
(array([-1.17683641e-16,  2.60902411e-17,  6.66133815e-18, -1.16989751e-16]),
 array([1., 1., 1., 1.]))
```

Question 9: Simulate traffic density (cars per minute) recorded by 6 sensors over 24 hours.

Task:

- Generate array shape (24, 6)
- Find hourly average
- Identify the **busiest sensor** (highest total density)
- Apply a threshold:
    - Replace all values above 50 with 50 (traffic cap)
- Compute energy usage:

energy = Σ(sensor_density × 0.5)

Code:

```
import numpy as np

traffic = np.random.randint(0, 101, (24,6))

hourly_avg = np.mean(traffic, axis=1)
busiest_sensor = np.argmax(np.sum(traffic, axis=0))

traffic_capped = np.where(traffic > 50, 50, traffic)
energy = np.sum(traffic_capped * 0.5)

hourly_avg, busiest_sensor, energy
```

Output:

```
(array([58.83333333, 39.        , 41.16666667, 43.83333333, 42.5       ,
        41.16666667, 36.33333333, 37.83333333, 43.83333333, 57.5       ,
        37.5       , 44.66666667, 56.66666667, 52.5       , 58.5       ,
        43.33333333, 49.66666667, 40.83333333, 59.83333333, 43.83333333,
        32.83333333, 33.33333333, 50.16666667, 74.5       ]),
 np.int64(4),
 np.float64(2552.0))
```

Question 10: You are simulating forces in a mechanical system.

Force vectors applied to 5 components:

F = [[5,2,1],

[3,1,4],

[6,3,2],

[4,2,5],

[7,1,3]]

Transformation matrix:

T = [[2,1,0],

[1,3,1],

[0,2,2]]

Task:

- Compute transformed forces: F_new = F × T
- Compute magnitude of each force vector
- Identify the component experiencing highest transformed force
- Perform eigenvalue decomposition of T

Code:

```python
import numpy as np

F = np.array([
    [5,2,1],
    [3,1,4],
    [6,3,2],
    [4,2,5],
    [7,1,3]
])

T = np.array([
    [2,1,0],
    [1,3,1],
    [0,2,2]
])

F_new = F @ T
magnitudes = np.linalg.norm(F_new, axis=1)
highest_force_component = np.argmax(magnitudes)
eigenvalues, eigenvectors = np.linalg.eig(T)

F_new, magnitudes, highest_force_component, eigenvalues
```

Output:

```
(array([[12, 13,  4],
        [ 7, 14,  9],
        [15, 19,  7],
        [10, 20, 12],
        [15, 16,  7]]),
 array([18.13835715, 18.05547009, 25.19920634, 25.37715508, 23.02172887]),
 np.int64(3),
 array([4.30277564, 2.        , 0.69722436]))
```

Lab 04:Seaborn and scikit learn

PART A: DATA VISUALIZATION USING SEABORN

Q1. Patient Stay Analysis (Healthcare)

You are given a dataset containing age, disease_type, and hospital_stay_days.

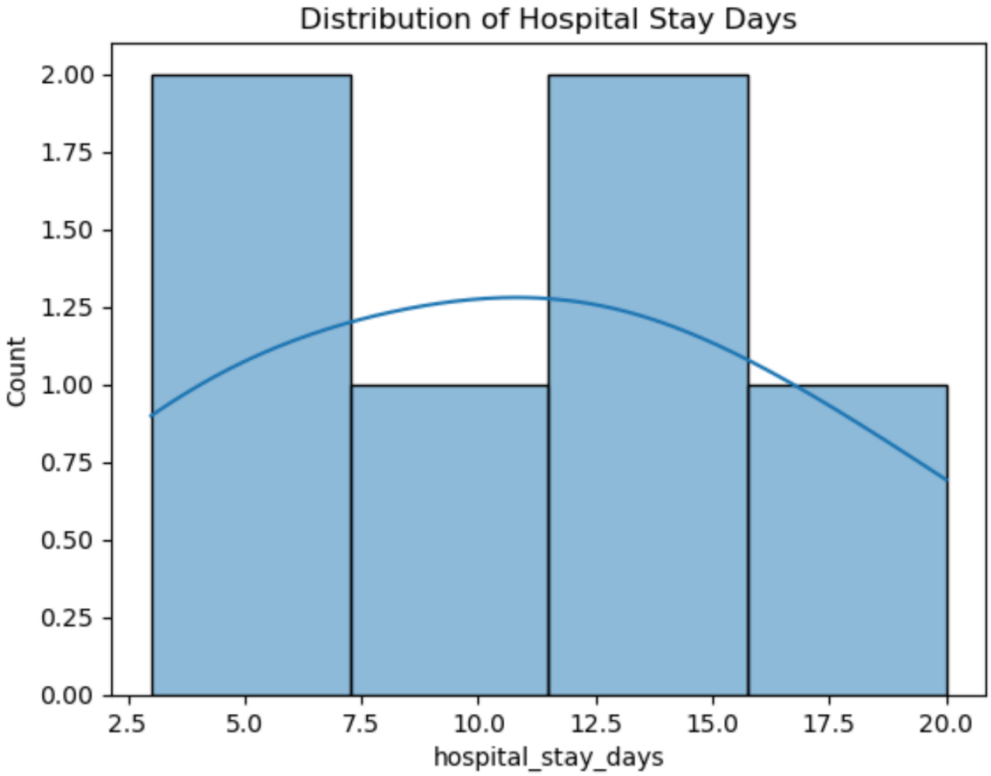
Write a Python program using **Seaborn** to:

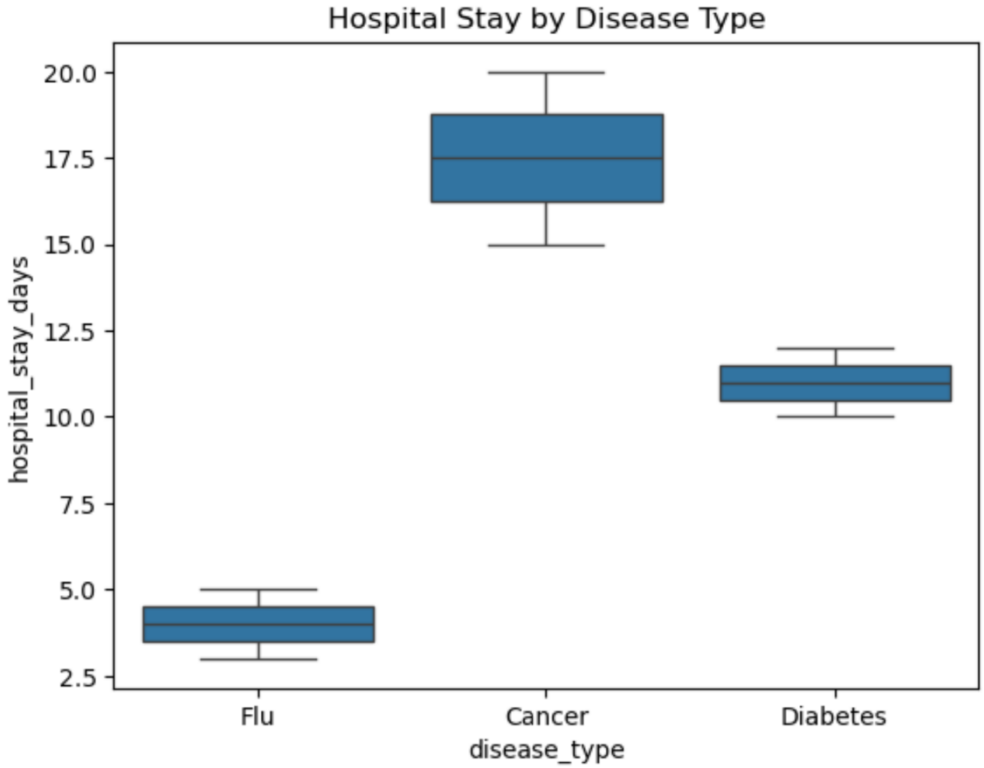
- Plot the distribution of hospital stay days
- Compare hospital stay across different disease types
- Identify which disease has the highest median stay

Code:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Sample dataset
data = {
    'age': [25, 40, 60, 30, 50, 70],
    'disease_type': ['Flu', 'Cancer', 'Cancer', 'Flu', 'Diabetes', 'Diabetes'],
    'hospital_stay_days': [3, 15, 20, 5, 10, 12]
}

df = pd.DataFrame(data)

# Distribution plot
sns.histplot(df['hospital_stay_days'], kde=True)
plt.title("Distribution of Hospital Stay Days")
plt.show()

# Comparison across disease types
sns.boxplot(x='disease_type', y='hospital_stay_days', data=df)
plt.title("Hospital Stay by Disease Type")
plt.show()

# Highest median stay
print(df.groupby('disease_type')['hospital_stay_days'].median())
```

Output:


Distribution of Hospital Stay Days
Count
hospital_stay_days
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0


Hospital Stay by Disease Type
hospital_stay_days
disease_type
Flu
Cancer
Diabetes
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

```
disease_type
Cancer      17.5
Diabetes    11.0
Flu          4.0
Name: hospital_stay_days, dtype: float64
```

Q2. Sales Distribution Comparison (Business)

A retail dataset contains region and monthly_sales.

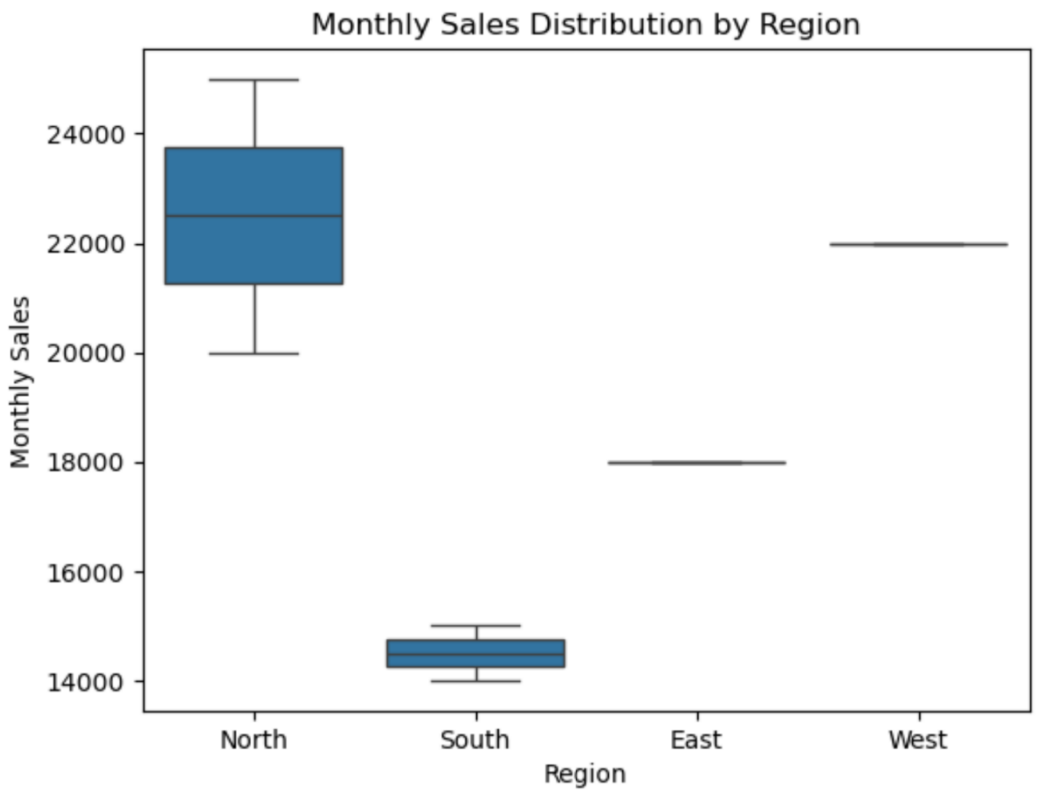
Write code to:

- Visualize sales distribution per region using Seaborn
- Detect regions with high sales variability
- Label the axes and add a title

Code:

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

data = {
    'region': ['North', 'South', 'East', 'West', 'North', 'South'],
    'monthly_sales': [20000, 15000, 18000, 22000, 25000, 14000]
}

df = pd.DataFrame(data)

sns.boxplot(x='region', y='monthly_sales', data=df)
plt.title("Monthly Sales Distribution by Region")
plt.xlabel("Region")
plt.ylabel("Monthly Sales")
plt.show()
```

Output:



Q3. Student Performance Correlation (Education)

A dataset contains study_hours, attendance, and final_score .

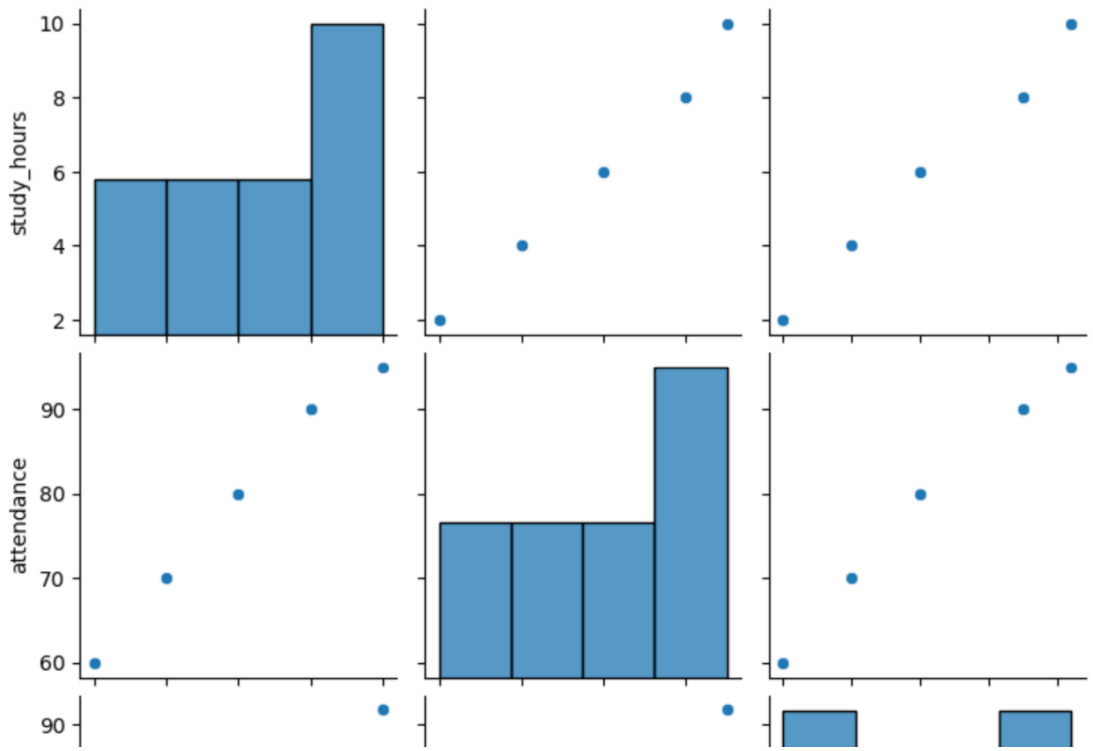
Using Seaborn:

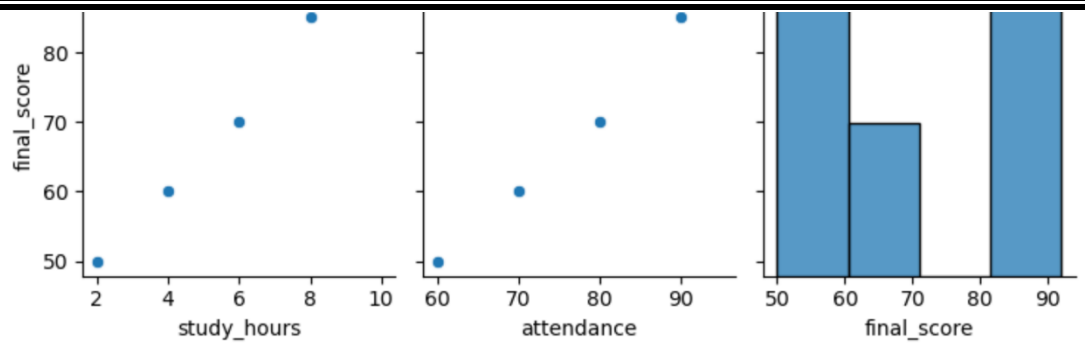
- Create a pair plot
- Plot a correlation heatmap
- Interpret which feature most affects the final score

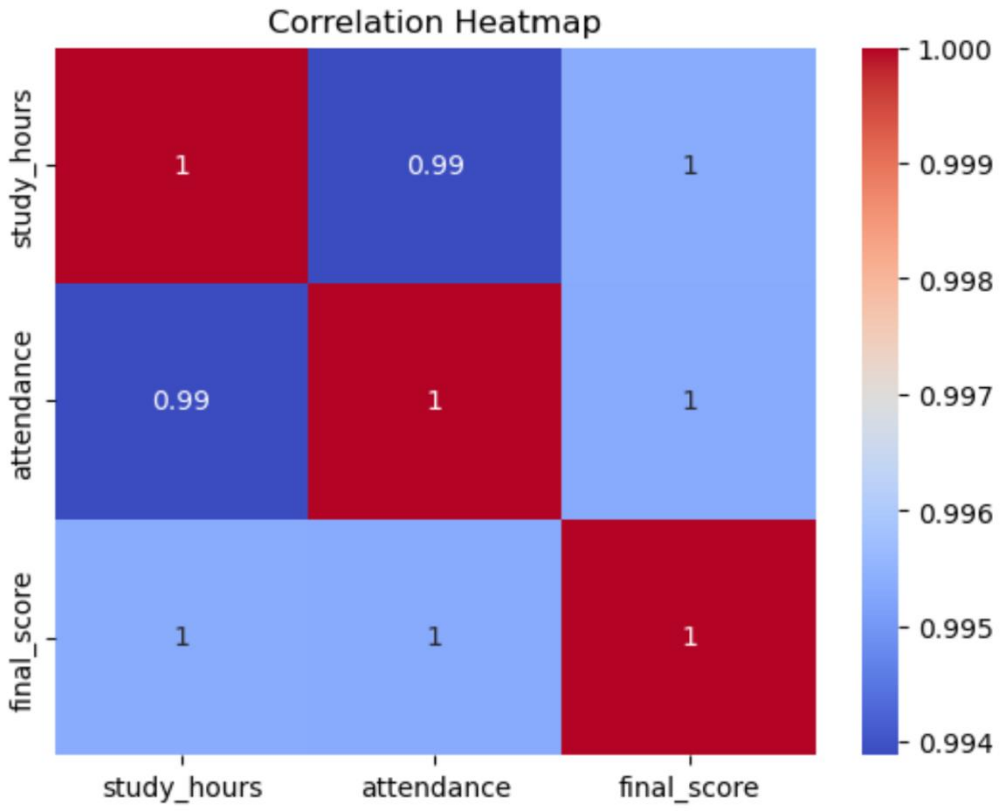
Code:

```python
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

data = {
    'study_hours': [2, 4, 6, 8, 10],
    'attendance': [60, 70, 80, 90, 95],
    'final_score': [50, 60, 70, 85, 92]
}

df = pd.DataFrame(data)

# Pair plot
sns.pairplot(df)
plt.show()

# Heatmap
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```

Output:







Q4. Customer Segmentation Visualization

Customer data includes age, annual_income, and spending_score.

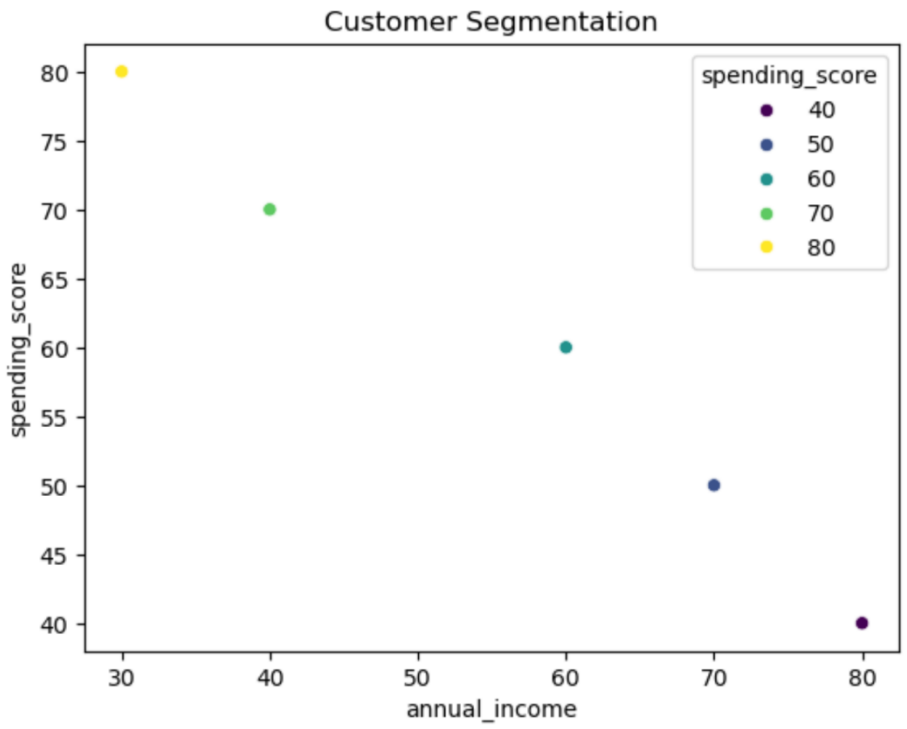
Write a program to:

- Visualize relationships using Seaborn scatter plots
- Color-code points based on spending score
- Identify potential customer segments visually

Code:

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

data = {
    'age': [20, 30, 40, 25, 35],
    'annual_income': [30, 60, 80, 40, 70],
    'spending_score': [80, 60, 40, 70, 50]
}

df = pd.DataFrame(data)

sns.scatterplot(
    x='annual_income',
    y='spending_score',
    hue='spending_score',
    palette='viridis',
    data=df
)
plt.title("Customer Segmentation")
plt.show()
```

Output:



Q5. Model Result Visualization

You trained a classifier and obtained y_true and y_pred.

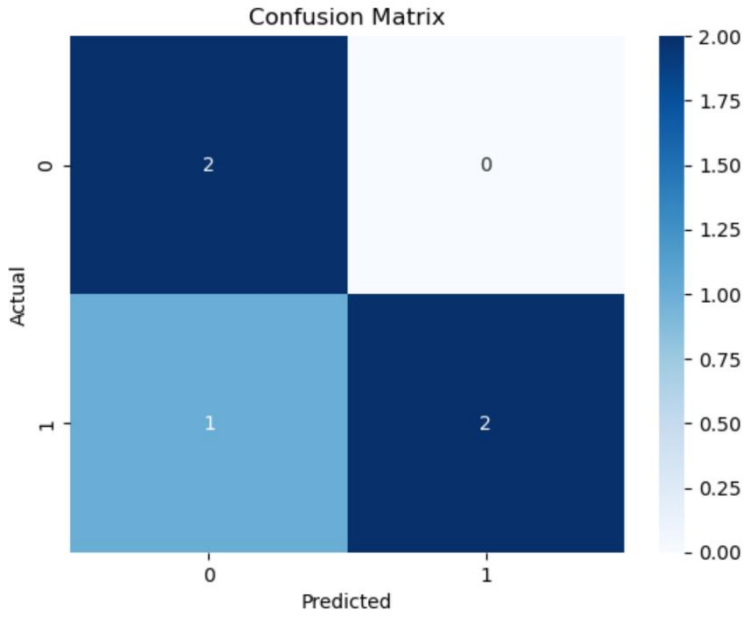
Write code using Seaborn to:

- Plot a confusion matrix heatmap
- Add annotations and color bar
- Clearly label predicted vs actual classes

Code:

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix

y_true = [0, 1, 1, 0, 1]
y_pred = [0, 1, 0, 0, 1]

cm = confusion_matrix(y_true, y_pred)

sns.heatmap(cm, annot=True, cmap='Blues', fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

Output:



PART B: MACHINE LEARNING USING SCIKIT-LEARN

Q1. Telecom Customer Churn Prediction

Given customer data with features such as tenure, monthly_charges, and contract_type, and target churn.
Write a Scikit-learn program to:

- Encode categorical variables
- Train a Logistic Regression model
- Evaluate using accuracy and confusion matrix

Code:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

data = {
    'tenure': [1, 12, 24, 6],
    'monthly_charges': [50, 70, 90, 60],
    'contract_type': ['Month-to-month', 'One year', 'Two year', 'Month-to-month'],
    'churn': [1, 0, 0, 1]
}

df = pd.DataFrame(data)

le = LabelEncoder()
df['contract_type'] = le.fit_transform(df['contract_type'])

X = df.drop('churn', axis=1)
y = df['churn']

model = LogisticRegression()
model.fit(X, y)

pred = model.predict(X)

print("Accuracy:", accuracy_score(y, pred))
print("Confusion Matrix:\n", confusion_matrix(y, pred))
```

Output:

```
Accuracy: 1.0
Confusion Matrix:
 [[2 0]
 [0 2]]
```

Q2. House Price Prediction

A dataset contains area, bedrooms, location_score, and price.

Implement a regression model to:

- Split data into train/test sets
- Train Linear Regression
- Predict prices and calculate Mean Squared Error

Code:

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import pandas as pd

data = {
    'area': [1000, 1500, 2000, 2500],
    'bedrooms': [2, 3, 3, 4],
    'location_score': [7, 8, 9, 9],
    'price': [200000, 300000, 400000, 500000]
}

df = pd.DataFrame(data)

X = df.drop('price', axis=1)
y = df['price']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25)

model = LinearRegression()
model.fit(X_train, y_train)

pred = model.predict(X_test)
print("MSE:", mean_squared_error(y_test, pred))
```

Output:

```
MSE: 0.039999839968673 76
```

Q3. Disease Classification System

Patient data includes glucose, BMI, age, and outcome.

Write code to:

- Normalize the data
- Train a Decision Tree classifier
- Evaluate precision, recall, and F1-score

Code:

```python
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report
import pandas as pd

data = {
    'glucose': [120, 140, 160, 180],
    'BMI': [25, 30, 28, 35],
    'age': [30, 45, 50, 60],
    'outcome': [0, 1, 1, 1]
}

df = pd.DataFrame(data)

X = df.drop('outcome', axis=1)
y = df['outcome']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

model = DecisionTreeClassifier()
model.fit(X_scaled, y)

pred = model.predict(X_scaled)
print(classification_report(y, pred))
```

Output:

```
                precision    recall  f1-score   support

           0       1.00      1.00      1.00         1
           1       1.00      1.00      1.00         3

    accuracy                           1.00         4
   macro avg       1.00      1.00      1.00         4
weighted avg       1.00      1.00      1.00         4
```

Q4. Email Spam Detection

You are given email text and spam labels.

Using Scikit-learn:

- Convert text to numerical features (TF-IDF)
- Train a Naive Bayes classifier
- Test the model on new email text

Code:

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

emails = ["Win money now", "Meeting tomorrow", "Free prize"]
labels = [1, 0, 1]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(emails)

model = MultinomialNB()
model.fit(X, labels)

test_email = vectorizer.transform(["Free money"])
print("Prediction:", model.predict(test_email))
```

Output:

```
Prediction: [1]
```

Q5. Student Risk Prediction

Student records include attendance, mid_marks, and final_marks.

Write a program to:

- Build a Random Forest classifier
- Predict whether a student is “At Risk”
- Display feature importance

Code:

```
from sklearn.ensemble import RandomForestClassifier
import pandas as pd

data = {
    'attendance': [60, 80, 90, 50],
    'mid_marks': [40, 70, 85, 35],
    'final_marks': [45, 75, 90, 40],
    'risk': [1, 0, 0, 1]
}

df = pd.DataFrame(data)

X = df.drop('risk', axis=1)
y = df['risk']

model = RandomForestClassifier()
model.fit(X, y)

print("Feature Importance:", model.feature_importances_)
```

Output:

```
Feature Importance: [0.37777778 0.38888889 0.23333333]
```

Lab 05:Tensorflow in python

Learning Objectives / Outcomes:

- To understand the **TensorFlow library**, its purpose in machine learning and deep learning, and its key features such as multi-platform support, scalability, and visualization with TensorBoard.
- To learn the concept of **tensors, operations, and computational graphs**, and how TensorFlow executes computations using eager execution or sessions.

What is TensorFlow?

TensorFlow is an **open-source library developed by Google** for machine learning (ML) and deep learning (DL). It is widely used to build and train neural networks for tasks like image recognition, natural language processing, and predictive modeling.

Key Features of TensorFlow:

- **Multi-platform:** Runs on CPU, GPU, mobile devices, and even browsers.
- **Flexible:** Supports high-level APIs like Keras for easy model building.
- **Scalable:** Can handle large datasets and distributed training.
- **Visualization:** Comes with TensorBoard for monitoring and visualizing model performance.

Why is it called TensorFlow?

- **Tensor** → a fancy name for multi-dimensional data (like arrays, matrices).
- **Flow** → means data flows through different layers of a neural network.

So **TensorFlow = data (tensors) flowing through operations**.

Where is TensorFlow used?

TensorFlow is used in:

- Image recognition
- Face detection
- Voice assistants
- Text classification
- Chatbots
- Medical analysis
- Recommendation systems (like Netflix)
- Self-driving cars

```python
import tensorflow as tf
from tensorflow import keras

# Simple model with one hidden layer
model = keras.Sequential([
    keras.layers.Dense(10, activation='relu'),
    keras.layers.Dense(1)
])

# Compile the model
model.compile(optimizer='adam', loss='mse')

# Dummy data
x = [1, 2, 3, 4]
y = [2, 4, 6, 8]

# Train the model
model.fit(x, y, epochs=50)

# Predict
print(model.predict([5]))
```

2. How TensorFlow Works

TensorFlow works with **tensors**, which are just **multi-dimensional arrays**. It builds a **computation graph**, where each node represents a mathematical operation, and edges represent data (tensors) flowing between operations.

Key Components:

1. **Tensors:** Basic data structure (like arrays or matrices).
   Example: Scalars (0D), Vectors (1D), Matrices (2D), etc.
2. **Operations (Ops):** Mathematical operations applied to tensors (like addition, multiplication).
3. **Computational Graph:** A graph representing the flow of data through operations.
4. **Sessions (TF1.x) / Eager Execution (TF2.x):**
   - TensorFlow 1.x used sessions to run graphs.
   - TensorFlow 2.x executes operations immediately (like regular Python), making it easier to debug

Implementation

```
# Import necessary libraries
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Prepare Data
# Features (X) and labels (Y)
X = np.array([1, 2, 3, 4, 5], dtype=float)
Y = np.array([2, 4, 6, 8, 10], dtype=float)  # Linear relationship: y = 2*x

# Step 2: Build the Model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(units=1, input_shape=[1])  # 1 input, 1 output
])

# Step 3: Compile the Model
model.compile(
    optimizer='sgd',               # Stochastic Gradient Descent optimizer
    loss='mean_squared_error'      # Loss function
)

# Step 4: Train the Model
history = model.fit(X, Y, epochs=1000, verbose=0)  # Train silently

# Step 5: Make Predictions
new_X = np.array([6, 7, 8], dtype=float)
predictions = model.predict(new_X)
print("Predictions for [6, 7, 8]:", predictions.flatten())

# Step 6: Visualize the Results
plt.scatter(X, Y, color='blue', label='Original Data')  # Original points
plt.plot(X, model.predict(X), color='red', label='Fitted Line')  # Regression line
plt.scatter(new_X, predictions, color='green', label='Predictions')  # Predictions
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Linear Regression using TensorFlow")
plt.legend()
plt.show()
```

Output:

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 56ms/step
Predictions for [6, 7, 8]: [11.986539  13.9809065 15.975274 ]
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 53ms/step
```